Ders 11 Genişletilmiş Notu: Kuyruk Türleri, Ağaçlar, Ağaçta Gezinme ve İkili Arama Ağacı

Ders 11 Genişletilmiş Notu: Kuyruk Türleri, Ağaçlar, Ağaçta Gezinme ve İkili Arama Ağacı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kuyruk Türleri
- 2. Doğrusal Veri Yapılarından Doğrusal Olmayanlara
- 3. Ağaç Terminolojisi
- 4. Ağaçların Kullanım Alanları
- 5. İkili, Üçlü ve n'li Ağaçlar
- 6. Ağaçta Gezinme (Tree Traversal)
- 7. Özyinelemeli Gezinme Fonksiyonları
- 8. İkili Ağaç Türleri
- 9. Dengeli, Dejenere ve Yaslı Ağaçlar
- 10. İkili Arama Ağacı (Binary Search Tree)
- 11. BST'de Arama
- 12. BST'ye Ekleme: Diziden Ağaç Kurma
- 13. BST'den Silme
- 14. Kendini Dengeleyen Ağaçlar
- Örnek Sorular
- Dersin Toparlaması

1. Kuyruk Türleri

Geçen hafta kuyruklarda (queue) kalınmıştı. Hoca hem konuyu toparlamak hem de geçen hafta değinilmeyen noktaları tamamlamak için kuyruk türlerini sıraladı. Dairesel kuyruk (circular queue) daha önce konuşulmuş ve gerçeklenmişti. Bu derste dört tür daha ele alındı:

- Girişi kısıtlı kuyruk (input restricted queue)
- Çıkışı kısıtlı kuyruk (output restricted queue)
- Çift uçlu kuyruk (double ended queue, deque)
- Öncelikli kuyruk (priority queue)

Normal bir kuyrukta bir uçtan ekleme (enqueue), öbür uçtan çıkarma (dequeue) yapılır. Kuyruk ilk giren ilk çıkar (FIFO, first in first out) mantığıyla çalışır: sıradaki kişi hizmet alır, yeni gelen sona eklenir.

1.1 Girişi Kısıtlı Kuyruk

Hoca önce öğrencilere bu kuyruğun ne tür bir özelliği olabileceğini sordu. Cevap gelmeyince kendisi açıkladı. Girişi kısıtlı kuyrukta ekleme yalnız bir uçtan yapılır, ama çıkarma her iki uçtan yapılabilir. Yani normal kuyruğa ek olarak, en son eklenen eleman da kuyruktan çıkarılabilir.

Bunun amacı şudur: Kuyruk normalde FIFO mantığıyla çalışsa da bazı durumlarda son eklenen elemanın da çıkarılması gerekebilir. Örneğin eleman çok uzun süre beklemiş olabilir. Hocanın örneği ağ tamponuydu (buffer): Gelen ağ paketleri tamponda bekler, bir fonksiyon sürekli sıradaki paketi işlemekle meşguldür. İşlem yavaş kalırsa tamponun dolma, kuyruğun tıkanma olasılığı doğar. Bu durumda sona eklenmiş paketleri düşürme yoluna gidilebilir. Bunu yapabilmek için kuyruğun ekleme yapılan ucundan da çıkarma yapılabilmesi gerekir.

1.2 Çıkışı Kısıtlı Kuyruk

Bu türde her iki uçtan eleman eklenebilir, ama yalnız bir uçtan eleman çıkarılabilir. Normalde elemanlar geldikçe sona eklenir. Çıkışı kısıtlı kuyrukta, elemanın çıkarıldığı uca, yani ilk hizmeti alacak noktaya da eleman eklenebilir.

Hocanın örneği: Sıradaki elemanı çıkarmak üzeresiniz ve daha yüksek öncelikli bir eleman geldi. Bu istisnai durumda (exceptional case) yeni eleman en başa eklenir. Böylece bazı özel durumlar kuyruğun kendi yapısı içinde yönetilebilir.

1.3 Çift Uçlu Kuyruk

Çift uçlu kuyruk, önceki iki yapının birleşimidir. Hem başlangıç noktasına eleman eklenir hem oradan eleman çıkarılır. Aynı şekilde kuyruğun son noktasına da eleman eklenir ve oradan da çıkarılır.

Hocaya göre bu türlerin dairesel kuyruktan temel farkı şudur: Kuyruğun bir ya da iki ucu, normal görevine ek bir görevle kontrol edilebilir hale getirilir ve bu imkân kullanıcıya verilir. Kullanıcı da kuyruğu bu iş için yazılmış fonksiyonlarla yönetir.

1.4 Öncelikli Kuyruk

Öncelikli kuyruk bir yığın (heap) yapısıyla kurulur. Bu bir ağaç yapısıdır. En üst noktada en yüksek ya da en düşük önceliğe sahip eleman bulunur. Elemanlar küçükten büyüğe ya da büyükten küçüğe bir düzene göre yerleşir. Hoca örnek olarak `1, 3, 5, 7, 6` ve tersi yönde `7, 6, 3, 5, 1` dizilimlerini verdi. Kuyruğun tepesinde her zaman ilk hizmeti alacak eleman, yani düzene göre en küçük ya da en büyük değer yer alır. Böylece öncelikli işler bu ağaç yapısında konumlanmış olur.

Hoca ayrıntıları gelecek hafta heap veri yapısıyla birlikte anlatacağını söyledi.

> **Ders sonrası not:** Hocanın verdiği iki dizilim tam sıralı değildir: ilkinde 7, 6'dan önce; ikincisinde 3, 5'ten önce gelir. Bu bir hata değildir, çünkü heap tam sıralama istemez, yalnız her ebeveynin çocuklarından küçük (ya da büyük) olmasını ister. 0 tabanlı dizi gösteriminde `1, 3, 5, 7, 6` dizilimi bu koşulu sağlar ($1 \leq 3$, 5 ve $3 \leq 7$, 6). `7, 6, 3, 5, 1` dizilimi de ters koşulu sağlar ($7 \geq 6$, 3 ve $6 \geq 5$, 1).

Hoca, yığın (stack) konusunda yapılan karşılaştırmayı hatırlattı: Kuyruğun değişik biçimleri yığınınkinden daha fazladır, çünkü gerçek hayatta kullanım alanları daha geniştir. Bu yapılar bu kullanım alanlarına uygun olarak tasarlanmıştır.

2. Doğrusal Veri Yapılarından Doğrusal Olmayanlara

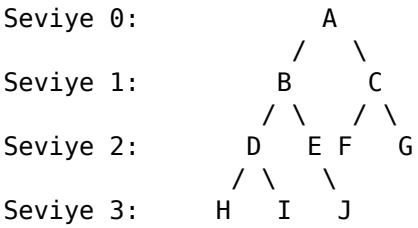
Şimdiye kadar görülen veri yapıları doğrusaldı (linear): dizi (array), bağlı liste (linked list), yığın ve kuyruk. Bunlar elemanları belirli bir düzene göre saklar. Elemanlar genellikle tek bir düz (flat) yapıda yan yana konumlanır. Bunlar basit yapılardır ve tek seviyeden oluşur. Bellek kullanımı açısından da çeşitli dezavantajları vardır.

Bundan sonra doğrusal olmayan (non-linear) veri yapıları gelecek. Bunlarda bir hiyerarşi vardır. Bilgiler belirli bir seviyelendirmeye göre, birbirleriyle ilişkili olarak saklanır.

> **Hocanın vurgusu:** Ağaç, grafın özel bir halidir. Her graf bir ağaç değildir, ama her ağaç bir graftır.

3. Ağaç Terminolojisi

Hoca, terimlerin bir kısmının ayrık matematik dersinden tanıdık olabileceğini, ama oturmamış bilgileri pekiştirmek için bir fırsat olduğunu söyledi. Terimler slayttaki şu örnek ağaç üzerinden anlatıldı (düğüm yerleri altyazıdaki anlatımdan kurulmuştur):



- **Kök (root):** Her ağacın bir kökü vardır. Örnekte kök A'dır.
- **Kenar (edge) ve düğüm (vertex, node):** Graflarda olduğu gibi bağlantılara kenar, noktalara düğüm denir. Ağaç veri yapılarında düğüm için çoğunlukla "node" kelimesi kullanılır.
- **İkili ağaç (binary tree) ile genel ağaç:** "Ağaç" dendiğinde belirli bir ağaç türü kastedilmez; bir düğümün n tane çocuğu olabilir. "İkili ağaç" dendiğinde ise her düğümün en fazla iki çocuğu vardır.
- **Seviye (level):** Kökün seviyesi 0'dır. B ve C seviye 1'de, D, E, F, G seviye 2'de, H, I, J seviye 3'tedir. Seviye, ağacın derinliği hakkında bilgi verir.
- **Yol (path):** Bir düğümden diğerine giderken üzerinden geçilen düğümlerdir. A ile J arasındaki yol A, B, E, J'dir.
- **Ebeveyn (parent):** H ve I'nın ebeveyni D'dir. J'nin ebeveyni E'dir.
- **Ata (ancestor):** Slaytta yer almayan, hocanın eklediği terim. J'nin ebeveyni E'dir. Onun üstündeki B ve onun da üstündeki A, J'nin atalarıdır. J'den köke kadar olan tüm düğümler J'nin atasıdır.
- **Çocuk (child):** D'nin çocukları H ve I'dır.
- **Alt ağaç (subtree):** Büyük ağacın altındaki küçük ağaçlardır. D, H, I bir alt ağaçtır. B, D, E, H, I, J başka bir alt ağaçtır.
- **Ziyaret (visiting):** Ağaçta gezinirken üzerinden geçilen düğüm ziyaret edilmiş olur.
- **Gezinme (traversing):** Ağacın üzerinde bir kurala göre gezinmektir. Temel olarak üç kural vardır: inorder, preorder ve postorder. Önce sol düğüm, sonra sağ düğüm, sonra ana düğüm mü; yoksa önce ana düğüm, sonra sol, sonra sağ mı? Hangisine ihtiyaç varsa ağaç ona göre gezilir.
- **Yaprak (leaf):** Hiç çocuğu olmayan düğümdür. Örnekte F, G, J, I ve H yapraktır.
- **Kardeş (sibling):** Aynı ebeveyne bağlı düğümlerdir. F ile G kardeştir, D ile E kardeştir. J'nin kardeşi yoktur. İkili ağaçta bir ebeveynin en fazla iki çocuğu olur. Genel ağaçta bu sayı n olabilir.
- **Orman (forest):** Birden fazla ağacın, birbirlerine bağlantıları olmadan bir arada bulunduğu yapıdır. Hoca, yapay zekada karar ağaçları (decision tree) ve rastgele orman (random forest) gibi sınıflandırma yöntemlerinde bu kavramla daha çok karşılaşılacağını söyledi.

Hoca ziyaret kavramını anlatırken "A, B, D, H, I, J, E" gibi bir gezinti örneği verdi.

> **Ders sonrası not:** "A, B, D, H, I, J, E" yalnız ziyaret kavramını gösteren bir örnektir, aşağıda anlatılan standart sıralardan biri değildir. Bu alt ağacın preorder sırası A, B, D, H, I, E, J olurdu.

4. Ağaçların Kullanım Alanları

Hoca ağaçların gerçek hayatta nerelerde kullanıldığını sıraladı:

- **Dosya sistemleri:** Dosyaların birbirleriyle ilişkisini kurmak ağacın en yoğun kullanıldığı yerlerden biridir. Linux'ta en tepede kök dizin (root directory) bulunur. Onun altında `home`, `etc` gibi klasörler, `home` altında kullanıcı klasörleri vardır. Dizin yapısı ve dosyaların saklanması bu mantıkla mümkün olur.
- **Huffman kodlaması:** Sıkıştırma yapılırken Huffman mantığıyla ağaçtan yararlanılır. Hoca bunun ayrık matematikte görülmüş olabileceğini söyledi.

- **Veritabanları:** Veriler saklanırken ve indekslenirken ağaçlar kullanılır.
- **Derleyiciler:** Derleme sırasında sözdizimi ağacı (syntax tree) oluşturulur.
- **Öncelikli kuyruk:** Heap veri yapısı bir ağaçtır.
- **Yapay zeka:** Karar ağacı, rastgele ağaç (random tree) ve rastgele orman gibi yöntemlerde, karar verici düğümler üzerinden bir yol izleyerek sonuca ulaşır. Hocanın örneği bir kişinin kadın mı erkek mi olduğunu ayırt etmekti. Her düğümde bir soru sorulur: saçı uzun mu, kaş, göz mesafesi, ağız çevresi, yanaklar gibi. Belirli bir yolun sonunda karar verilir.
- **Coğrafi bilgi sistemleri:** Alanlar büyüdükçe indeksleme gerekir. k-d ağaçları (k-d tree) gibi yapılarla, dünyadaki bir noktaya erişirken milyonlarca, milyarlarca seçeneği tek tek sorgulamak yerine önce hangi kıta, sonra hangi ülke, sonra hangi şehir gibi sorularla bir hiyerarşi kurulur. Böylece $O(n)$ mertebesindeki karmaşıklık, ikili aramada (binary search) olduğu gibi $O(\log n)$ mertebesine iner.

Hoca birçok ağaç türü olduğunu söyledi: ikili ağaç, ikili arama ağacı, AVL ağacı, heap ağacı ve diğerleri. Bunlardan yeri geldikçe bahsedilecek. İlk etapta ikili ağaç ve ikili arama ağacı ilgi noktasıdır.

5. İkili, Üçlü ve n'li Ağaçlar

Hoca gezinmeye geçmeden önce ağaçları çocuk sayısına göre ayırdı:

- **İkili ağaç (binary tree):** Her düğümün en fazla iki çocuğu vardır (at most two children).
- **Üçlü ağaç (ternary tree):** Her düğümün en fazla üç çocuğu vardır.
- **n'li ağaç (n-ary tree):** Çocuk sayısına bir sınır konmaz; bir ebeveynin n çocuğu olabilir.

Bu ayrım önemlidir. Bilgisayar biliminde problemler genelde ikili ağaçlar üzerinden çözülür. Ama ikili ağacın, hatta üçlü ağacın yetmediği, n'li ağaç gereken yapılar da sık karşınıza çıkar. Veriyi saklama noktasında, hem veritabanlarında hem dosya yapılarında bunlarla karşılaşılır. Derslerde veri işleme ağırlıklı sorular ikili ağaçlar üzerinden çözülecek. Hoca, günlük hayatta, örneğin haritalama işlemlerinde farklı çocuk sayısına sahip ağaçlar kullanmak gerekeceğini ekledi.

6. Ağaçta Gezinme (Tree Traversal)

Ağaçta gezinme temel ihtiyaçlardan biridir. Ağaç aşağıya doğru genişleyen bir yapıdır. Ağaç genişledikçe alt katmanlara ulaşmak için kullanılan özyinelemeli (recursive) fonksiyonlar belirli kısıtlarla karşılaşabilir. Bu yüzden teknik olarak ağacın çok derin olması istenmez.

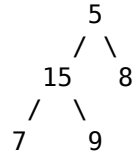
> **Hocanın vurgusu:** Ağacın bir tarafa yaslı (skewed) olması sevilmeyen bir durumdur. Mümkün olduğunca dengeli dağılmış bir ağaç üzerinde çalışılır. Dengeli ağacın en büyük artısı $O(n)$ karmaşıklığındaki işlemleri $O(\log n)$ karmaşıklığına indirmesidir. Ağaç bir tarafa yaslanmışsa $O(n)$ karmaşıklığına mahkûm olursunuz. Bu yüzden dengeli ağaçlar kullanılır ya da ağaçlar dengeli forma dönüştürülür.

Üç temel gezinme yöntemi vardır:

Yöntem	Sıra
Inorder	sol alt ağaç, düğüm, sağ alt ağaç
Preorder	düğüm, sol alt ağaç, sağ alt ağaç
Postorder	sol alt ağaç, sağ alt ağaç, düğüm

6.1 Slayttaki Örnek

Slayttaki örnek ağaç şudur:



Hoca bu ağaç üzerinde pek çok soru sorulabileceğini söyledi: Hangi düğümler çocuktur? Ağacın derinliği nedir? Kardeş düğüm var mıdır? Kökte ne vardır? Ağaç dengeli midir? 9'a ulaşmak için hangi yol izlenir? Ağaç büyüdükçe sorular daha karmaşık hale gelir. (Bu soruların cevapları Örnek Sorular bölümündedir.) Derste ilgilenilen kısım gezinmeydi.

Alt ağaç mantığı gözetilerek ilerlenir. 5'in sol alt ağacı 15, 7, 9'dur. Bu alt ağacın da sol tarafında 7 kalır.

Inorder: Önce en sol alt ağaca kadar inilir. 5'ten 15'e, oradan en alttaki 7'ye inilir. 7 alınır. Sonra o alt ağacın düğümü 15 alınır. Sonra sağ alt ağaca, 9'a geçilir. Sonra ortadaki kök 5'e çıkılır. En son sağ alt ağaçta yalnız 8 olduğu için 8 alınır. Sonuç: **7, 15, 9, 5, 8**.

Hoca burada sağ alt ağacın altında başka düğümler olsaydı ne olacağını da gösterdi: 9'un solunda 6, sağında 4 olsaydı, 15'ten sonra sağ alt ağaca gidildiğinde önce 6, sonra 9, sonra 4 alınırdı; yani dizi 7, 15, 6, 9, 4, 5, 8 olurdu.

Preorder: Önce düğümün kendisi alınır. İlk düğüm 5'tir. Sonra 15, 7, 9 alt ağacına geçilir. Bu alt ağacın ana düğümüyle (ebeveyniyle) başlanır: 15. Sonra sol çocuk 7, sonra sağ çocuk 9 alınır. Sonra yukarı dönülür ve 5'in sağ tarafındaki 8 alınır. Sonuç: **5, 15, 7, 9, 8**.

Postorder: Önce sol, sonra sağ, sonra düğüm gelir. 15, 7, 9 alt ağacına gidilir, oradan 7 alınır. Sonra sağdaki 9 alınır. Yukarı çıkılır, 15 alınır. 5'in sol tarafı bittiği için sağdaki kardeşe, 8'e geçilir. Düğüm en son geldiği için 5 en son alınır. Sonuç: **7, 9, 15, 8, 5**.

6.2 Seviye Seviye Gezinme

Slaytta iki seçenek daha vardı:

- **Yukarıdan aşağı, soldan sağa (from top, left to right):** Elemanlar seviye seviye üstten alta gezilir: **5, 15, 8, 7, 9**.
- **Aşağıdan yukarı, soldan sağa (from bottom, left to right):** Seviye seviye alttan üste gezilir: **7, 9, 15, 8, 5**.

Ağaç gezinme işlemlerinde ağırlıklı olarak inorder, preorder ve postorder tercih edilir. Hoca ileride graflarda derinlik öncelikli arama (depth first search) ve genişlik öncelikli arama (breadth first search) algoritmalarının da anlatılacağını söyledi.

7. Özyinelemeli Gezinme Fonksiyonları

Aradan sonra hoca şu soruyu sordu: Bu gezinme yöntemlerini gerçeklemek için hangi yaklaşım uygundur, yineli (iterative) mi özyinelemeli (recursive) mi? Bir öğrenci özyinelemeli cevabını verdi. Hoca da özyinelemeli gezinmenin daha akıllıca olduğunu söyledi.

> **Ders sonrası not:** Derste özyinelemeli gezinmenin "daha verimli (efficient)" olduğu söylendi. Özyineleme kodu kısaltır ve kolaylaştırır. Çalışma süresi yineli çözümle aynı mertebededir: her düğüm bir kez ziyaret edildiği için $O(n)$. Özyineleme ayrıca çağrı yığınında ağacın yüksekliği kadar alan kullanır.

Önce bir düğüm yapısı (structure) gerekir. Düğüm bir değer saklar; bu `data` alanında tutulur. İkili ağaçta her düğümün solunda ve sağında birer düğüm olabileceği için `struct node` tipinde `left` ve `right` adlı iki adres alanı tutulur.

Özyinelemeli yazıldığında kodlar çok kısa ve basittir. Fonksiyon çağrılırken kök düğümün adresi gönderilir. Kök `NULL` ise fonksiyon hemen döner ve sola ya da sağa dallanma olmaz. Ağaçtaki durma noktası budur: en son düğümün altındaki adresin `NULL` olduğu varsayılarak durulur. Bu, özyinelemeli fonksiyonun temel durumudur (base case).

Aşağıdaki program derste anlatılan yapıyı ve üç fonksiyonu, iki örnek ağaç üzerinde çalıştırır (`gcc -Wall` ile derlenip çalıştırıldı):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
```

```
    struct node *left;
    struct node *right;
};

struct node *yeni(int x) {
    struct node *p = malloc(sizeof(struct node));
    p->data = x;
    p->left = p->right = NULL;
    return p;
}

void inorder(struct node *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void preorder(struct node *root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void postorder(struct node *root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

void serbest(struct node *root) {
    if (root == NULL) return;
    serbest(root->left);
    serbest(root->right);
    free(root);
}

int main(void) {
    /* Ornek 1:  5 (sol 15, sag 8), 15 (sol 7, sag 9) */
    struct node *r1 = yeni(5);
    r1->left = yeni(15);
    r1->right = yeni(8);
    r1->left->left = yeni(7);
    r1->left->right = yeni(9);
    printf("Agac 1 inorder  : "); inorder(r1);  printf("\n");
    printf("Agac 1 preorder : "); preorder(r1); printf("\n");
    printf("Agac 1 postorder: "); postorder(r1); printf("\n");

    /* Ornek 2:  5 (sol 3, sag 7), 3 (sol 4, sag 2) */
    struct node *r2 = yeni(5);
    r2->left = yeni(3);
    r2->right = yeni(7);
    r2->left->left = yeni(4);
    r2->left->right = yeni(2);
    printf("Agac 2 inorder  : "); inorder(r2);  printf("\n");
    printf("Agac 2 preorder : "); preorder(r2); printf("\n");
    printf("Agac 2 postorder: "); postorder(r2); printf("\n");
```

```
    serbest(r1);
    serbest(r2);
    return 0;
}
```

Çıktı:

```
Agac 1 inorder  : 7 15 9 5 8
Agac 1 preorder : 5 15 7 9 8
Agac 1 postorder: 7 9 15 8 5
Agac 2 inorder  : 4 3 2 5 7
Agac 2 preorder : 5 3 4 2 7
Agac 2 postorder: 4 2 3 7 5
```

`yeni`, `serbest` ve `main` fonksiyonları programı çalıştırmak için eklendi; derste gösterilen kısım yapı ve üç gezinme fonksiyonudur.

7.1 Inorder Fonksiyonunun Adım Adım İzi

Hoca fonksiyonların nasıl çalıştığını tablette çizdiği şu ağaç üzerinde izledi (bu ağaç bir ikili ağaçtır, ikili arama ağacı değildir):

```
      5
     / \
    3   7
   / \
  4   2
```

- `inorder(5)` çağrılır. 5 `NULL` mı? Hayır. Fonksiyon kendini sol çocukla çağırır.
- `inorder(3)`: `NULL` değil. Yine sol çocukla çağrılır.
- `inorder(4)`: `NULL` değil. 4'ün solu çağrılır, `NULL` döner. 4 yaprak olduğu için temel durum sağlanmıştır. Ekrana ilk olarak **4** yazılır. 4'ün sağında bir şey yoktur, oradan da `NULL` döner.
- 3'e dönülür. **3** yazılır. 3'ün sağı çağrılır: 2. 2'nin altında bir şey yoktur, **2** yazılır.
- 3'ün solu ve sağı bitmiştir. Bir üst çağrıya dönülür; elde 5 vardır. **5** yazılır.
- 5'in sağına bakılır: 7. İki tarafı da `NULL` olduğu için **7** yazılır ve en üst çağrı da biter.

Sonuç: 4, 3, 2, 5, 7.

Preorder'da temel fark fonksiyon çağrılarıyla `printf` satırının yerinin değişmesidir. Inorder'da en soldaki düğüm önce yazılacağı için en sola gidilir ve oradan ilk çıkılan noktada değer yazılır. Preorder'da önce düğüm, sonra çocuklar yazıldığı için fonksiyona ilk girildiği anda kökün değeri yazdırılır. 5 ile başlanır, sonra 3, sonra 4, sonra 3'ün sağındaki 2, en son 7 yazılır: 5, 3, 4, 2, 7.

Postorder'da düğüm en son yazılır. Önce dallanma yapılır, fonksiyon kendini çağırır. 5'ten girilir, sola 3'e, tekrar sola 4'e gidilir. Önce 4 yazılır. Sonra 3'ün sağına gidilir ve 2 yazılır. 4 ve 2 yazıldıktan sonra sıra düğümün kendisindedir: 3. Sonra 7, en son 5 yazılır: 4, 2, 3, 7, 5.

> **Hocanın vurgusu:** Gezinmeyi özyinelemeli kurgulamak çok daha kolaydır. Yineli çözmeye çalışırsanız daha karmaşık bir kod yazmanız ve daha karmaşık bir algoritma tasarlamanız gerekir. Ama ağaç derinleştikçe bu çağrılar birikir; çok derin bir ağaçta gezinme yığın taşmasıyla (stack overflow) sonlanabilir.

Hoca ağaç terminolojisinin ve gezinmenin büyük kısmının ayrık matematikte görüldüğünü öğrencilerden teyit etti. Bu konuların ilerleyen haftalarda sınavlarda ve ödevlerde sık karşılarına çıkacağını, anlaşılmayan bir yer olursa sormaktan çekinmemelerini söyledi.

8. İkili Ağaç Türleri

Hoca tanımları tablete İngilizce olarak yazdı.

- **İkili ağaç (binary tree):** Her düğümün en fazla iki çocuğu (at most two children) vardır.
- **Dolu ikili ağaç (full binary tree):** Her düğümün 0 ya da 2 çocuğu vardır (every node has 0 or 2 children). Bir ağacın dolu olup olmadığına çocuk sayılarına bakarak karar

verilir. Hoca, verilen bir ağacın ikili ağaç olup olmadığını kontrol etmenin de ayrı bir iş olduğunu ekledi.

- **Mükemmel ikili ağaç (perfect binary tree):** Tüm iç düğümlerin (internal node, yani yaprak olmayan düğümler) iki çocuğu vardır ve tüm yapraklar aynı seviyededir (all internal nodes have two children and all leaf nodes are at the same level).
- **Tam ikili ağaç (complete binary tree):** Son seviye dışındaki tüm seviyeler tamamen doludur. Son seviyede tam doluluk beklenmez, ama var olan düğümler sola yaslı ve aralarında boşluk kalmayacak şekilde, kesintisiz dizilmelidir.

Hoca slayttaki örnekte düğümlerin sola yaslı olduğunu gösterdi. Düğümler `2, 3, 4, 5, 6` biçiminde, son seviyede arada boşluk bırakarak yerleşmiş olsaydı tam ikili ağaçtan söz edilemezdi.

8.1 Dört Örnek Üzerinde Dolu ve Tam Ağaç

Hoca slayttaki dört örnek ağaç üzerinden iki sınıflandırmayı birlikte yaptı:

Örnek	Gözlem	Dolu (full)	Tam (complete)
1	3'ün tek çocuğu var; son seviye sola yaslı değil, arada boşluk var	Hayır	Hayır
2	Her düğümün 0 ya da 2 çocuğu var; son seviye sola yaslı değil	Evet	Hayır
3	2'nin tek çocuğu var, ama bu çocuk en solda; diğer seviyeler dolu	Hayır	Evet
4	Her düğümün 0 ya da 2 çocuğu var; sola yaslı, kesintisiz, ara seviyeler dolu	Evet	Evet

Birinci örnekte hoca önce ağacın neden dolu olmadığını sordu. Cevap: 3'ten tek bir dallanma var, yani bir düğüm tek çocuğa sahip; dolu ağaçta çocuk sayısı 0 ya da 2 olmalıdır. Tam ağaç olmamasının sebebi de bir öğrencinin "sağda olmaz" cevabıyla ifade edildi: 5 sola yaslı değildir ve arada boşluk vardır.

9. Dengeli, Dejenere ve Yaslı Ağaçlar

9.1 Dengeli İkili Ağaç

Hoca ağacın dengeli olmasının çok önemli olduğunu tekrarladı. Temel sebep, dengenin işlemlerin karmaşıklığını $\log n$ seviyesine indirmesidir. Slaytın sağ tarafındaki örnek dengesiz bir ağaçtı, ama bir tarafa (sağa ya da sola) yaslı olmadığı için nispeten kabul edilebilir bir ağaçtı. Bir tarafa yaslı olma durumu en sevilmeyen durumdur, çünkü karmaşıklığı $O(n)$ seviyesine geri taşır. O zaman diziden bir farkı kalmaz ve hiyerarşik yapı verimli kullanılmamış olur.

Hoca sonra bir ağacın dengeli olup olmadığına nasıl karar verildiğini sordu. Öğrencilerden gelen cevaplar ve hocanın düzeltmeleri şöyleydi:

- "Dalların yüksekliklerinin eşit olması gerekir" cevabına hoca iyi bir nokta ama tam doğru değil dedi.
- "Sağ yaprakla sol yaprağın seviyesi eşit olmalı" cevabına seviyelerin eşit olması ifadesinin doğru olmadığını söyledi.
- "Fark birden büyük olmayacak" cevabını doğru buldu. Bire bir eşitlik beklenmez; bir fark olabilir. Fark birden büyük olursa ağaç dengesizdir (unbalanced).

Slayttaki tanım şuydu: “the difference between the left and right subtree for any node is not more than one”, yani herhangi bir düğümde sol ve sağ alt ağaçlar arasındaki fark birden fazla değildir.

Hoca iki örnek gösterdi. Birinde 3’ün bulunduğu dal ile 2, 4, 5 alt ağacının bulunduğu dal arasında bir seviye fark vardı; kurala uygun olduğu için dengeli ağaçtı. Diğerinde 3 seviye 1’de kalırken 2, 4, 5, 6 alt ağacındaki 6 düğümü seviye 3’teydi; aradaki fark 2 olduğu için dengesiz ağaçtı.

> **Ders sonrası not:** Hoca tanımı açıklarken alt ağaçların “boyutundan” söz etti. Burada ölçüt düğüm sayısı değil, alt ağaçların yüksekliğidir. Koşul yalnız kökte değil, ağacın her düğümünde sağlanmalıdır.

Hoca dengesiz ama yaslı olmayan bir ağacın daha iyi hale getirilebileceğini, ama bunun büyük bir problem olmadığını söyledi. Bir ağacı gerçekten dönüştürme zahmetine girmeye değip değmeyeceği tartışılır. Bir ağacın hangi noktada dengeli ağaca dönüştürülmesi gerektiğine karar vermek de ayrı ve önemli bir konudur.

9.2 Dejenere ve Yaslı Ağaçlar

- **Dejenere ağaç (degenerate tree) ya da patolojik ağaç (pathological tree):** Her iç düğümün tek bir çocuğu vardır.
- **Yaslı ikili ağaç (skewed binary tree):** Dejenere ağacın özel halidir. Bütün düğümler tamamen sola ya da tamamen sağa yaslıdır. Hocanın örneği 1, 2, 3 (ve isteğe bağlı 4) düğümlerinin hep aynı tarafa bağlanmasıydı. Sağ tarafa 7, 8, 9 gibi dizilmiş bir ağaç sağa yaslı ikili ağaçtır (right skewed binary tree).

Yaslı ağaçlar aynı zamanda patolojik ağaçtır, çünkü bu tanımı kapsarlar. Hoca bunların hiç istenmeyen iki senaryo olduğunu ve mümkün olduğunca uzak durulması gerektiğini söyledi.

10. İkili Arama Ağacı (Binary Search Tree)

İkili arama ağacı (BST), verileri bir hiyerarşi gözeterek ve belirli bir düzende saklamayı sağlayan önemli bir yapıdır.

> **Hocanın vurgusu:** İkili ağaç ile ikili arama ağacı farklıdır. İkili arama ağacı ile ikili arama (binary search) algoritması da birbirinden farklı iki kavramdır.

Temel kurallar:

- Sol alt ağaçtaki tüm düğümler kök düğümden küçüktür (all the nodes of the left subtree are less than the root node).
- Sağ alt ağaçtaki tüm düğümler kök düğümden büyüktür (all the nodes of the right subtree are greater than the root node).

Slayttaki örnekte kök 8’di. Sol alt ağaçtaki 5, 6, 4, 1 değerlerinin hepsi 8’den küçüktü. Hoca ağaca yeni elemanlar ekleseydi nereye gideceklerini gösterdi: 15, 10 ve 14 gibi değerler sağ alt ağaçta uygun yerlere konumlanırdı, çünkü sağ alt ağaçtaki değerler her zaman 8’den büyüktür.

Hoca, veriler üzerinde ne yapılabiliyorsa ağaçtaki veriler üzerinde de aynı işlerin yapılabilmesi gerektiğini söyledi. BST’de en kolay işlemler arama (search) ve eklemedir (insert). Silme (delete) daha zordur.

> **Ek bilgi (derste anlatılmadı):** BST kuralı yüzünden bir BST’nin inorder gezinmesi, elemanları küçükten büyüğe sıralı verir. Örneğin 12. bölümde kurulan ağacın inorder gezinmesi 5, 10, 12, 15, 20, 30, 35, 40’tır. Silme işlemindeki “sağ alt ağacın en küçüğü” ve “sol alt ağacın en büyüğü” kavramları da bu sıralı dizide silinen elemanın hemen sonraki ve hemen önceki komşularıdır.

11. BST’de Arama

Hoca arama yaparken nelere dikkat edilmesi gerektiğini, durma koşulunun ne olduğunu sordu. Cevap gelmeyince kendisi açıkladı.

Arama yapıldığında aranan eleman ağaçta ya vardır ya yoktur; durumlar bu ikisidir. İlk olarak kök düğüme bakılır: aranan elemana eşit mi? Değilse sola mı sağa mı gidileceğine karar verilir. Solda küçükler, sağda büyükler yer aldığı için karşılaştırmaya göre yön seçilir.

Slayttaki ağaçta 26 aranırken izlenen yol:

Adım	Düğüm	Karşılaştırma	Karar
1	15	26 > 15	sağa git
2	21	26 > 21	sağa git
3	27	26 < 27	sola git
4	25	26 > 25	sağa git

11.1 Durma Koşulu

Durma noktası veri yapısına bağlıdır:

- Ağaç bir dizide saklanıyorsa, eleman bulunmayan gözler başta -1 yapılır. -1'den farklı olduğu sürece aramaya devam edilir.
- Ağaç özyinelemeli, kendine başvuran bir yapıyla (self-referencing structure) saklanıyorsa, bir sonraki düğümün `NULL` olup olmadığı kontrol edilir.

Eleman bulunursa da arama durdurulmalıdır. Adım sayısı belli olmayan bir iş olduğu için `while` döngüsü kullanılır.

11.2 Dizi Üzerinde Arama Akışı

Slayttaki akış diyagramında döngü, `tree[i]` -1'den farklı olduğu ve `x`'e eşit olmadığı sürece devam ediyordu. Döngüde `x`'in o gözdeki elemana göre küçük mü büyük mü olduğuna bakılır. Küçükse sola, büyükse sağa gidilir.

Sola ve sağa gidişin formülü, kökün dizinin 0. gözüne mi 1. gözüne mi konduğuna bağlıdır. Slaytta $2i + 1$ ve $2i + 2$ kullanılmıştı. Hoca bunun 0'dan başlayan bir yapı olduğunu sordu, öğrenciler doğruladı. Kök 1. göze konursa formüller değişir.

> **Hocanın vurgusu:** Diziyi 0'dan da 1'den de başlatabilirsiniz, ama indisleri değiştirirken ona göre bir kurgu yapmanız gerekir. Varsayımınızın ne olduğunu her zaman söylemelisiniz.

C dilinde 0'dan başlayan dizi düşünülürse, örneğin 9 aranırken: kökte 15 vardır, 9 < 15 olduğu için doğru (true) koluna gidilir ve $i = 2 \cdot 0 + 1 = 1$ olur. 1. gözde 9 bulunur.

Döngüden çıkıldığında kontrol edilmesi gereken şey, içinde değer olan bir göze mi yoksa boş bir göze mi varıldığıdır. -1'e varıldıysa "bulunamadı" (not found) denir. Gerçek bir değer bulunduysa, kaçıncı gözde olduğu yazdırılarak elemanın var olduğu bildirilir.

12. BST'ye Ekleme: Diziden Ağaç Kurma

Hoca silmeye geçmeden önce eklemeyi konuşmak istedi, çünkü silme daha zordur. Elde şöyle bir dizi olsun:

```
20, 10, 30, 5, 15, 40, 12, 35
```

Bu normal bir dizidir. Elemanları tek tek, kurallara uygun şekilde bir BST'ye yerleştirmek istiyoruz. Nasıl bir akış diyagramı çizilir? Hoca öğrencilere 5-6 dakika kağıt kalemle akış diyagramı çizmeleri için süre verdi.

Bir öğrenci "özyinelemeli olarak yeni düğümü koyacağımız yeri bulmalıyız" dedi. Hoca bunun olabileceğini ama çok genel bir ifade olduğunu söyledi ve diyagramı birlikte kurmaya başladı:

- n ve $A[n]$ dışarıdan alınır.
- Ağaç için `tree` adlı bir dizi kullanılır. Boyutu sembolik olarak büyük bir sayı, örneğin 1000 seçilir. Gözlerin hepsi başta -1 yapılır.
- Dizideki elemanları tek tek yerleştirmek için bir döngü gerekir.

- Her eleman için ağaçta, `tree[i]` -1'e eşit olmadığı sürece ilerlenir. Eleman o gözdeki değerden büyükse sağa, değilse sola gidilir. Bu kısım aramadakiyle aynıdır.
- Döngüden çıkıldığında boş bir göze varılmıştır; eleman oraya yerleştirilir.

Hoca en kötü durumda kaç gözün -1 yapılması gerektiğini de söyledi: 2^n.

> **Ders sonrası not:** n elemanlı bir BST, elemanlar sıralı geldiğinde tamamen tek tarafa yaslanır. Sağa yaslı ağaçta en derin düğümün indisi $2^n - 2$, sola yaslı ağaçta $2^{n-1} - 1$ olur (python ile $n = 1, \ldots, 7$ için hesaplandı). Yani $2^n - 1$ göz yeter. Sonuç yine üsteldir: dizi gösterimi dengesiz ağaçlarda çok fazla göz israf eder.

12.1 İlk Taslaktaki Hatalar

Diyagramın ilk hali tamamlandığında hoca çözümde problemli bir kısım olup olmadığını sordu. Hatalar adım adım bulundu:

- **Dizi C'de 0'dan başlar.** Bir öğrenci, taslağın "Pascal formatına göre doğru" olduğunu ama C için dizinin 0'dan başlaması gerektiğini söyledi. Hoca döngü başlangıcını 0 yaptı ve sınırın da $n - 1$ olması gerektiğini ekledi.
- **Aynı indis iki dizi için kullanılıyordu.** Bir öğrenci, `i` her seferinde artıyorsa `tree[i]`'nin nasıl -1'e eşit olmayacağını sordu. Hoca, burada iki yapı olduğunu, ikisinde de gezildiğini ve temel hatanın ne olduğunu sordu. Bir öğrenci doğru cevabı verdi: Girdi dizisinin indisi, ağaç indisi gibi kullanılıyordu. Hoca girdi dizisinin indisini `j` yaptı; dizinin elemanı `A[j]` olmalıydı.

  > **Hocanın vurgusu:** Aynı indisi hem girdi dizisi hem ağaç dizisi için kullanmak "algoritmanın kalbine bıçak saplamak", bir "algoritma cinayeti"dir. Böyle bir kullanım olamaz; olursa kod tamamen yanlış çalışır. Hoca bunun birinci sınıf seviyesinde bir hata olduğunu, çok daha önce fark edilmesini beklediğini söyledi.
- **Ağaç indisi her eleman için kökten başlatılmıyordu.** İlk döngüden çıkıldıktan sonra sonraki elemanda `i` sıfırlanmıyordu. Hoca, iç döngüye girmeden önce `i = 0` yapılması gerektiğini gösterdi.
- **Sol ve sağ formülleri ters yazılmıştı.** İzleme sırasında 10 yerleştirilirken `i`'nin 2 olduğu görüldü. Oysa `10 < 20` olduğu için sola, yani $2i + 1 = 1$ numaralı göze gidilmeliydi. Büyükse $2i + 2$, değilse $2i + 1$ olmalıdır. Hoca bunun basit görünse de her şeyin yanlış çalışmasına sebep olabileceğini vurguladı.

Ters formüllerle kurulan ağaçta 10, 2. göze; 30, 1. göze gider. Yani ağaç aynalanır ve küçükler sağda, büyükler solda kalır; bu BST kuralına aykırıdır (python ile denendi).

12.2 Düzeltilmiş Kod ve İz

Derste birlikte düzeltilen akışın C karşılığı ve dizi üzerinde arama aşağıdadır (`gcc -Wall` ile derlenip çalıştırıldı):

```c
#include <stdio.h>

#define SIZE 1000

int tree[SIZE];

/* Dizideki n elemani 0 tabanli dizi gosterimli BST'ye yerlestirir. */
void diziden_agac(int a[], int n) {
    int i, j, k;
    for (k = 0; k < SIZE; k++)
        tree[k] = -1;
    for (j = 0; j < n; j++) {          /* girdi dizisinin indisi: j */
        i = 0;                          /* her eleman icin kokten basla */
        while (tree[i] != -1) {         /* agac dizisinin indisi: i */
            if (a[j] > tree[i])
                i = 2 * i + 2;          /* sag cocuk */
            else
```

```
                i = 2 * i + 1;          /* sol cocuk */
        }
        tree[i] = a[j];
    }
}

/* Bulursa gozun indisini, bulamazsa -1 dondurur. */
int ara(int x) {
    int i = 0;
    while (tree[i] != -1 && tree[i] != x) {
        if (x < tree[i])
            i = 2 * i + 1;
        else
            i = 2 * i + 2;
    }
    if (tree[i] == -1)
        return -1;
    return i;
}

int main(void) {
    int a[] = {20, 10, 30, 5, 15, 40, 12, 35};
    int n = sizeof(a) / sizeof(a[0]);
    int k, x;
    int aranan[] = {12, 35, 26};

    diziden_agac(a, n);
    for (k = 0; k < SIZE; k++)
        if (tree[k] != -1)
            printf("tree[%d] = %d\n", k, tree[k]);
    for (k = 0; k < 3; k++) {
        x = ara(aranan[k]);
        if (x == -1) printf("%d: not found\n", aranan[k]);
        else         printf("%d: %d. gozde bulundu\n", aranan[k], x);
    }
    return 0;
}
```

Çıktı:

```
tree[0] = 20
tree[1] = 10
tree[2] = 30
tree[3] = 5
tree[4] = 15
tree[6] = 40
tree[9] = 12
tree[13] = 35
12: 9. gozde bulundu
35: 13. gozde bulundu
26: not found
```

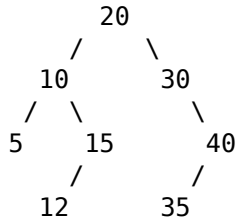
> **Ders sonrası not:** Bu kodda (derste çizilen akışta da) `i` değerinin dizi boyutunu aşıp aşmadığı kontrol edilmez. Ağaç çok derinleşirse `tree[i]` dizinin dışına taşar. Gerçek bir programda döngü koşuluna `i < SIZE` kontrolü eklenmelidir. Bu örnekte en büyük indis 13 olduğu için sorun çıkmaz.

Hoca izi ilk üç eleman için tahtada yaptı; kalan elemanlar aynı kuralla yerleşir (ara adımlar eklendi):

j	A[j]	Ziyaret edilen gözler (karşılaştırma)	Yerleştiği göz
0	20	`tree[0]` boş	0
1	10	0: 10 > 20 değil, sola	1
2	30	0: 30 > 20, sağa	2
3	5	0: sola; 1: 5 > 10 değil, sola	3
4	15	0: sola; 1: 15 > 10, sağa	4
5	40	0: sağa; 2: 40 > 30, sağa	6
6	12	0: sola; 1: sağa; 4: 12 > 15 değil, sola	9
7	35	0: sağa; 2: sağa; 6: 35 > 40 değil, sola	13

Hocanın tahtadaki anlatımı şöyleydi: $j = 0$ iken $A[0] = 20$, $i = 0$ ve `tree[0]` -1'dir. -1'den farklı olmadığı için döngüye girilmez, 20 kök olarak 0. göze konur. $j = 1$ iken $A[1] = 10$, i yine 0'dan başlar. `tree[0]` artık 20 olduğu için döngüye girilir. 10, 20'den büyük değildir; yanlış (false) koluna gidilir ve $i = 1$ olur. `tree[1]` -1 olduğu için döngüden çıkılır ve 10 oraya konur. $j = 2$ iken 30 gelir, $i = 0$'dan başlar. 30 > 20 olduğu için doğru koluna gidilir, $i = 2$ olur. `tree[2]` boş olduğu için 30 oraya konur. Bu iş 8 eleman için, $j = 0$'dan 7'ye kadar tekrarlanır.

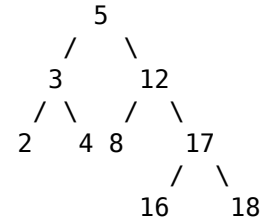
Ortaya çıkan ağaç:



Hoca, yapılan işin aramanın bir tık değiştirilmiş hali olduğunu söyledi. Burada bütün dizi yerleştirildi; tek bir elemanı yerleştiren bir ekleme fonksiyonu da yazılabilirdi.

13. BST'den Silme

Silme, ekleme ve arama kadar basit değildir; birkaç durumla karşılaşılır. Hoca şu ağacı çizdi:



Burada BST kurallarına uygun bir ağaçtan söz edildiğini, alelade bir ikili ağaç olmadığını vurguladı. Amaç elemanı çıkarırken kuralları korumaktır.

13.1 Durum 1: Yaprak Silme

Silinecek eleman bir yapraksa yapılacak özel bir şey yoktur. Örneğin 2 silinmek isteniyor: Yerine -1 yazılır ve 2 ağaçtan silinmiş olur. Ağacın yapısı bozulmaz.

13.2 Durum 2: Tek Çocuklu İç Düğüm

Silinecek düğüm bir iç düğümse ve tek çocuğu varsa ne yapılır? Hoca örnek istedi; bir öğrenci 17'yi söyledi. Hoca bunun için 18'in olmadığını varsaydı. 2 silindiği için 3'ün de artık tek çocuğu (4) olduğunu, onun da örnek olabileceğini ekledi.

17 silinmek istendiğinde 16, 17'nin yerine çıkar ve 16'nın eski gözüne -1 yazılır. Ağacın yapısı yine bozulmaz. Bu da teknik olarak zor değildir.

> **Ders sonrası not:** Dizi gösteriminde bu kolaylık yalnız taşınan çocuk yapraksa geçerlidir. Çocuğun kendi alt ağacı varsa, bütün alt ağaç bir seviye yukarı taşınmalıdır,

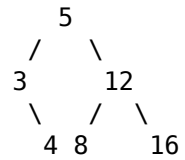
çünkü her düğümün indisi değişir. Bağlı (işaretçili) yapıda ise yalnız ebeveynin işaretçisi değiştirilir.

13.3 Durum 4: Eleman Ağaçta Yok

Hoca kolay olduğu için dördüncü durumu da araya aldı: Eleman ağaçta yoksa, arama sırasında -1'e kadar gidilmiştir. Bu durumda elemanın ağaçta olmadığı söylenir. Durum 1, 2 ve 4 rahatça yönetilir.

13.4 Durum 3: İki Çocuklu İç Düğüm

En büyük sıkıntı üçüncü durumdadır: silinecek düğüm hem iç düğümdür hem de iki çocuğu vardır. 2 ve 17 silindikten sonra ağaç şöyledir (5, 3, 12, 4, 8, 16):



Burada 12 silinmek isteniyor. Bir öğrenci 8'in 12'nin yerine geçebileceğini söyledi. Hoca bunun olabileceğini, 16'nın yanında 8'in yukarı çıkmasının da mümkün göründüğünü söyledi. Bir öğrenci kökü silmeye çalışınca işin karışacağını söyledi; hoca bunu doğruladı.

Bir öğrencinin önerisi genel kuralı verdi: Sağ alt ağaçtaki en küçük düğüm silinen düğümün yerine alınır. Bu düğüm sağ alt ağaçta hep sola (left'ten left'e) gidilerek bulunur. Kök 5 silinmek istenirse sağ alt ağacın en küçüğü 8'dir; 8 köke geçer ve ağacın düğümleri 8, 12, 16, 3, 4 olur.

Hoca bunu yeni bir örnek üzerinde denedi. Ağacın kökü 15'ti ve 20 silinmek isteniyordu. 20'nin sol alt ağacında 18 ve 16, sağ alt ağacında 32, 25 ve 38 bulunuyordu.

Öğrenci tekrar kuralı söyledi: Sağ alt ağaçta sürekli sola gidilirse, ulaşılan düğüm silinen düğümün yerine konabilir. Bu örnekte o düğüm 25'tir. 25 alınıp 20'nin yerine konduğunda, yani sağ alt ağacın en alt kısmındaki sol eleman yerine konduğunda, eleman silinirken ağacın yapısı korunur.

Hoca bir karşı örnek de sordu: 25 yerine 16 konsaydı ne olurdu? Kural bozulurdu. Sol tarafta kalan 18, 16'dan küçük olması gerekirken büyük olurdu.

> **Hocanın vurgusu:** BST'den silmenin en sıkıntılı durumu iki çocuklu düğümün silinmesidir. Silinen düğümün yerine sağ alt ağaçtaki en küçük eleman konur.

Hoca bir de şunu ekledi: Silme işlemi ağacın dengeli kalacağını garanti etmez. İşlemin sonunda ağacın dengesiyle ilgili bir sorun çıkarsa, bir dengeleme algoritması çalıştırılıp ağaç yeniden düzenlenmelidir.

13.5 Sol Alt Ağacın En Büyüğü Kullanılabilir mi?

Bir öğrenci, sol alt ağacın en büyüğünün de yerine konup konamayacağını sordu (bu örnekte 18). Hoca bildiği kadarıyla her durumda olmadığını söyledi. Bunu göstermek için ağacı değiştirerek bir durum kurmaya çalıştı ve 20 silinirken yerine 32'nin gelmesi gerektiğini söyledi; sonra yine "sağ alt ağaçtaki en küçük eleman alınır" dedi.

Başka bir öğrenci de sol alt ağacın en büyüğünün kullanılabileceğini söyledi. Hoca, bu özel örnekte işe yarıyor gibi göründüğünü ama her durumda doğruluğu garanti etmediğini düşündüğünü, şu an karşı örneği kuramadığını söyledi. Bunu öğrencilerin kontrol etmesini istedi ve gelecek hafta net cevabı vereceğini söyledi.

> **Ders sonrası not:** Sol alt ağacın en büyük elemanı (inorder öncül, inorder predecessor) da her durumda doğru sonuç verir. Bu eleman sol alt ağaçtaki diğer bütün değerlerden büyüktür ve sağ alt ağaçtaki bütün değerlerden küçüktür, yani silinen düğümün yerine konduğunda BST kuralı korunur. Bu, rastgele kurulmuş 20.000 BST'de rastgele bir eleman hem öncül hem ardıl yöntemiyle silinerek python ile denendi; hiçbir silmede kural bozulmadı. Ardıl (inorder successor) ya da öncül seçimi

bir tercih meselesidir. 12'li örnekte öncülle 8, ardılla 16 yukarı çıkar ve iki sonuç da geçerli BST'dir.

Ders sonrası not: Sağ alt ağacın en küçüğü her zaman yaprak değildir; sol çocuğu olamaz ama sağ çocuğu olabilir. Bu eleman silinen düğümün yerine taşındıktan sonra, sağ çocuğu onun eski yerine bağlanmalıdır (yani ardılın kendi silinmesi Durum 1 ya da Durum 2'dir).

Derste silme için kod yazılmadı. Aşağıdaki program, anlatılan dört durumu bağlı yapıda gerçekler ve dersteki 5'li ağaç üzerinde sırasıyla 2, 17, (ağaçta olmayan) 9 ve 5'i siler (`gcc -Wall` ile derlenip çalıştırıldı). Ağaç dersteki gibi 18'siz kuruldu.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left;
    struct node *right;
};

struct node *ekle(struct node *root, int x) {
    if (root == NULL) {
        root = malloc(sizeof(struct node));
        root->data = x;
        root->left = root->right = NULL;
    } else if (x < root->data) {
        root->left = ekle(root->left, x);
    } else {
        root->right = ekle(root->right, x);
    }
    return root;
}

struct node *en_kucuk(struct node *p) {   /* hep sola git */
    while (p->left != NULL)
        p = p->left;
    return p;
}

struct node *sil(struct node *root, int x) {
    struct node *tmp;
    if (root == NULL) {                   /* durum 4: eleman yok */
        printf("%d agacta yok\n", x);
        return NULL;
    }
    if (x < root->data) {
        root->left = sil(root->left, x);
    } else if (x > root->data) {
        root->right = sil(root->right, x);
    } else if (root->left == NULL) {      /* durum 1 ve 2 */
        tmp = root->right;
        free(root);
        return tmp;
    } else if (root->right == NULL) {     /* durum 2 */
        tmp = root->left;
        free(root);
        return tmp;
    } else {                              /* durum 3: iki cocuk */
        tmp = en_kucuk(root->right);      /* inorder ardil */
        root->data = tmp->data;
```

```
        root->right = sil(root->right, tmp->data);
    }
    return root;
}

void inorder(struct node *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void preorder(struct node *root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void serbest(struct node *root) {
    if (root == NULL) return;
    serbest(root->left);
    serbest(root->right);
    free(root);
}

int main(void) {
    int a[] = {5, 3, 12, 2, 4, 8, 17, 16};
    int n = sizeof(a) / sizeof(a[0]), k;
    struct node *root = NULL;

    for (k = 0; k < n; k++)
        root = ekle(root, a[k]);
    printf("Baslangic preorder: "); preorder(root); printf("\n");
    root = sil(root, 2);
    printf("2 silindi        : "); preorder(root); printf("\n");
    root = sil(root, 17);
    printf("17 silindi       : "); preorder(root); printf("\n");
    root = sil(root, 9);
    root = sil(root, 5);
    printf("5 silindi        : "); preorder(root); printf("\n");
    printf("inorder          : "); inorder(root); printf("\n");
    serbest(root);
    return 0;
}
```

Çıktı:

```
Baslangic preorder: 5 3 2 4 12 8 17 16
2 silindi        : 5 3 4 12 8 17 16
17 silindi       : 5 3 4 12 8 16
9 agacta yok
5 silindi        : 8 3 4 12 16
inorder          : 3 4 8 12 16
```

Son satırdaki inorder çıktısının sıralı olması, silmelerden sonra ağacın hâlâ BST olduğunu gösterir.

14. Kendini Dengeleyen Ağaçlar

Hoca son olarak öğrencilere bildikleri başka ağaç isimlerini sordu: ikili ağaç, ikili arama ağacı, n'li ağaç dışında daha "sofistike" isimler. Bir öğrenci B+ ağacını söyledi; hoca doğruladı. Hoca şu ağaçları tablete yazdı:

- **AVL ağacı:** Kendini dengeleyen (self-balancing) bir ikili arama ağacıdır.
- **Kırmızı-siyah ağaç (red-black tree):** Yine kendini dengeleyen bir BST'dir. Her düğüm için ekstra bir bit bilgi saklanır; bu bit düğümün kırmızı mı siyah mı olduğunu tutar. Ağacın dengede olup olmadığı bu bilgiler kullanılarak kontrol edilir.
- **B ağacı (B-tree):** Bu kez ikili olmayan, kendini dengeleyen bir ağaçtır. Erişim (access), ekleme (insertion) ve silme (deletion) işlemlerini verimli yapmayı gözeterek çalışır.
- **B+ ağacı:** Dosya sistemlerinde ve veritabanlarında kullanılır.

Hoca bunların ayrıntılarının bu dersin konusu olmadığını, ama bilgi sahibi olmanın faydalı olacağını, göz atmanızda yarar olduğunu söyledi.

Örnek Sorular

Bu derste ekran karesi yoktur. Aşağıdaki sorular derste sözlü olarak sorulan sorulardır.

1. (derste sözlü, 00:22:14) Girişi kısıtlı kuyruğun (input restricted queue) ne tür bir özelliği olabilir?

Çözüm: Ekleme yalnız bir uçtan yapılır, çıkarma ise iki uçtan da yapılabilir. Böylece FIFO düzenine ek olarak son eklenen eleman da çıkarılabilir. Örnek: ağ tamponunda çok bekleyen paketlerin, tıkanmayı önlemek için sondan düşürülmesi.

2. (derste sözlü, 00:47:22) Kökü 5, çocukları 15 ve 8, 15'in çocukları 7 ve 9 olan ağaçta: hangi düğümler çocuktur, ağacın derinliği nedir, kardeş düğüm var mıdır, kökte ne vardır, ağaç dengeli midir, 9'a ulaşmak için hangi yol izlenir? Ağacı inorder, preorder ve postorder gezin.

Çözüm: Gezinme derste yapıldı: inorder 7, 15, 9, 5, 8; preorder 5, 15, 7, 9, 8; postorder 7, 9, 15, 8, 5.

> **Çözüm (derste verilmedi):** Kök 5'tir. Çocuk olan düğümler 15, 8, 7 ve 9'dur (kökten başka herkes). Kök seviye 0 sayılırsa en derin düğümler 7 ve 9 seviye 2'dedir; ağacın derinliği 2'dir. Kardeşler 15 ile 8 ve 7 ile 9'dur. Ağaç dengelidir: kökte sol alt ağacın yüksekliği sağdakinden yalnız 1 fazladır, diğer düğümlerde fark 0'dır. 9'a giden yol 5, 15, 9'dur.

3. (derste sözlü, 01:02:56) Ağacı inorder, preorder ve postorder gezmek için hangi yaklaşımla kurgu yapmak uygundur: yineli mi, özyinelemeli mi?

Çözüm: Özyinelemeli. Kod çok kısa ve basittir; üç fonksiyon yalnız yazdırma satırının yeriyle ayrılır. Çok derin ağaçlarda yığın taşması riski vardır.

4. (derste sözlü, 01:17:50) Slayttaki ilk örnek ağacın dolu ikili ağaç (full binary tree) olmasını engelleyen nedir? Tam ikili ağaç (complete binary tree) olmamasının sebebi nedir?

Çözüm: 3'ün tek çocuğu vardır; dolu ağaçta her düğümün 0 ya da 2 çocuğu olmalıdır. Son seviyedeki düğüm (5) sola yaslı değildir ve arada boşluk vardır; bu yüzden tam ağaç da değildir.

5. (derste sözlü, 01:21:14) Bir ağacın dengeli ya da dengesiz olduğuna nasıl karar verilir?

Çözüm: Her düğümde sol ve sağ alt ağaçların yükseklik farkı en fazla 1 olmalıdır. Eşitlik beklenmez; fark 2 ve üzeri olursa ağaç dengesizdir.

6. (derste sözlü, 01:27:09) BST'de bir eleman aranırken nelere dikkat edilmelidir, durma koşulu nedir?

Çözüm: Eleman ya vardır ya yoktur. Kökten başlanır; eşitse bulunmuştur, küçükse sola, büyükse sağa gidilir. Dizi gösteriminde boş göz (-1) görülünce, bağlı gösterimde `NULL` görülünce durulur ve eleman yoktur. Eleman bulununca da durulur.

7. (derste sözlü, 01:30:45) Akış diyagramında sol ve sağ çocuk için $2i + 1$ ve $2i + 2$ kullanılmıştır. Dizinin 0'dan başladığı varsayılmış mıdır?

Çözüm: Evet. Kök 0. gözdeyse sol çocuk $2i + 1$, sağ çocuk $2i + 2$'dir. Kök 1. göze konursa formüller ona göre değişir; varsayım her zaman belirtilmelidir.

8. (derste sözlü, 01:34:11) `20, 10, 30, 5, 15, 40, 12, 35` dizisinin elemanlarını, BST kurallarına uygun olarak dizi gösterimli bir ağaca tek tek yerleştiren akış diyagramını çizin. Çizilen ilk çözümde hangi hatalar vardır?

Çözüm: Çözüm 12. bölümdedir. Elemanlar 0, 1, 2, 3, 4, 6, 9 ve 13. gözlere yerleşir. İlk taslaktaki hatalar: döngü C için 0'dan başlamalı ve $n - 1$'de bitmeli; girdi dizisi ve ağaç dizisi için ayrı indis (`j` ve `i`) kullanılmalı; her eleman için `i = 0` yapılmalı; büyükse $2i + 2$, değilse $2i + 1$ olmalı.

9. (derste sözlü, 02:05:53) 5 kökli örnek ağaçta, tek çocuklu bir iç düğüme örnek verin ve onu silin.

Çözüm: 18 yokken 17 (ya da 2 silindikten sonra 3). 17 silinince tek çocuğu 16 onun yerine çıkar.

10. (derste sözlü, 02:09:01) 5, 3, 12, 4, 8, 16 ağacında iki çocuklu 12'yi (ve kökü) silmek için ne yapılır?

Çözüm: Sağ alt ağacın en küçüğü (hep sola gidilerek bulunur) silinen düğümün yerine konur. Kök 5 silinirken bu eleman 8'dir.

> **Çözüm (derste verilmedi):** 12 silinirken sağ alt ağacın en küçüğü 16'dır; 16 yukarı çıkar ve ağaç 5, 3, 4, 16, 8 olur (inorder 3, 4, 5, 8, 16). Derste öğrencinin önerdiği 8'in çıkması da (sol alt ağacın en büyüğü) geçerli bir BST verir. İkisi de python ile doğrulandı.

11. (derste sözlü, 02:13:34) Kökü 15 olan örnekte 20 silinirken yerine 25 yerine 16 konsaydı olur muydu?

Çözüm: Hayır. 20'nin sol alt ağacındaki 18, yeni düğüm 16'dan büyük olurdu; sol alt ağacın kökten küçük olma kuralı bozulurdu.

12. (derste sözlü, 02:15:19) İki çocuklu düğüm silinirken sol alt ağacın en büyük elemanı da yerine konabilir mi?

Çözüm: Hoca derste her durumda olmadığını düşündüğünü söyledi ve kontrolü gelecek haftaya bıraktı.

> **Çözüm (derste verilmedi):** Evet, her durumda konabilir. Sol alt ağacın en büyüğü, sol alt ağacın geri kalanından büyük, sağ alt ağacın tümünden küçüktür; BST kuralı korunur. Rastgele 20.000 denemede öncül ile silme hiçbir kez kuralı bozmadı (python).

13. (derste sözlü, 02:18:47) İkili ağaç, ikili arama ağacı ve n'li ağaç dışında hangi ağaç türlerini biliyorsunuz?

Çözüm: AVL ağacı, kırmızı-siyah ağaç, B ağacı ve B+ ağacı. İlk ikisi kendini dengeleyen BST'dir; B ağacı kendini dengeleyen çok çocuklu bir ağaçtır; B+ ağacı dosya sistemlerinde ve veritabanlarında kullanılır.

Dersin Toparlaması

Ders, kuyruk konusunun kapanışıyla başladı. Girişi kısıtlı, çıkışı kısıtlı, çift uçlu ve öncelikli kuyrukların, kuyruğun uçlarına ek görevler vererek gerçek hayattaki özel durumları (paket düşürme, öncelikli elemanı öne alma) yönetmeyi sağladığı görüldü. Ardından doğrusal olmayan veri yapılarına geçildi. Ağacın grafın özel bir hali olduğu, kök, seviye, yol, ata, alt ağaç, yaprak, kardeş ve orman gibi terimler ve dosya sistemlerinden coğrafi bilgi sistemlerine ağaçların kullanım alanları anlatıldı. Inorder, preorder ve postorder gezinmenin özyinelemeli olarak üç satırlık fonksiyonlarla yazılabildiği, farkın yalnız yazdırma satırının yerinde olduğu iki örnek üzerinde izlendi.

İkinci yarıda ikili ağaç türleri (dolu, mükemmel, tam, dengeli, dejenere, yaslı) ve dengenin önemi işlendi: dengeli ağaçta işlemler $O(\log n)$, yaslı ağaçta $O(n)$'dir. İkili arama ağacında arama ve ekleme dizi gösterimi üzerinden akış diyagramıyla kuruldu; ekleme taslağındaki

indis hataları birlikte düzeltildi. Silmede dört durum ayrıldı ve en zor durum olan iki çocuklu düğüm için sağ alt ağacın en küçüğünün kullanıldığı gösterildi. Ders, kendini dengeleyen AVL, kırmızı-siyah, B ve B+ ağaçlarının tanıtılmasıyla bitti. Gelecek hafta heap veri yapısı, öncelikli kuyruk ve vakit kalırsa heap sort anlatılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.